

# LeetCode 1155: Number of Dice Rolls With Target Sum

Algorithm Analysis & Dynamic Programming Implementation

© Authored by Arpit Raj, LNMIIT Jaipur | July 12, 2026

## Problem Context & Overview

Tata 1mg

Snapchat

**Goal:** We are given  $n$  dice, where each die has a face range from  $1$  to  $k$ . The objective is to find the total number of possible ways to achieve an exact target  $sum$ .

### Combinatorics Insight:

The total number of ways to roll a die is  $k \times k \times k \dots; n$  times. This translates mathematically to  $k^n$  possible combinations. However, out of these  $k^n$  ways, we only require the subsets where the summation of the faces equals the target.

## Why Dynamic Programming?

---

Dynamic Programming is the ideal choice for this problem due to three main factors identified in the conceptual analysis:

- **Count number of ways:** We need to aggregate the total valid sequences dynamically.
- **Sequence of choices:** For each die rolled, we have a clear sequence of multiple choices (ranging from **1** to **k**).
- **Target sum:** The overlapping subproblems revolve around remaining dice and a constantly diminishing target.

*Note on Brute Force:* Attempting to iterate through every single die yields a time complexity of  $O(k^n)$ , which is computationally impossible for large inputs.

## DP Formulation & State Transitions

---

- **State Representation:**  $dp[d][sum]$  dictates the number of ways to successfully obtain the target sum utilizing precisely  $d$  dices.
- **Base Case:**  $dp[0][0] = 1$  (Using 0 dice to get a sum of 0 leaves exactly 1 valid state).
- **Stop Conditions:**
  - If  $d == 0$  and  $sum == 0 \rightarrow$  Return **1**
  - If  $d == 0$  and  $sum \neq 0 \rightarrow$  Return **0**
  - If  $sum < 0 \rightarrow$  Return **0** (It is impossible to reach a negative sum with positive integers).
- **Memoization Step:** Initialize the primary DP table with -1. During the execution of the solve function, whenever we encounter  $dp[d][sum] \neq -1$ , we immediately return the cached result.

- **Recurrence Relation:** To dynamically calculate the combinations for all faces, we aggregate using:

```
ways += solve(dice - 1, target - face)
```

## C++ Implementation

---

The code below implements the memoized Dynamic Programming solution. Indentation and styling are strictly maintained, with standard modulo arithmetic safely applied to avert integer overflow.

```

class Solution {
public:
    vector<vector<int>>> dp;
    const int MOD=1e9+7;
    int solve(int n, int k, int sum){
        if(n==0){
            if(sum==0){
                return 1; //base case dp[0][0]=1;
            }
            return 0;
        }

        if(sum<0) return 0;

        long long ways=0;
        if(dp[n][sum]!=-1) return dp[n][sum];

        for(int i=1;i<=k;i++){
            ways+=solve(n-1,k,sum-i);
            ways%=MOD;
        }

        dp[n][sum]=ways;
        return dp[n][sum];
    }
    int numRollsToTarget(int n, int k, int sum) {
        dp.assign(n+1,vector<int>(sum+1,-1));
        return solve(n,k,sum);
    }
};

```